Cómo el modelo C4 aporta claridad, estructura y una comprensión compartida a la arquitectura de software

PATROCINADO POR

¿POR QUÉ

EL MODELO C4?

¿QUÉ ES EL MODELO C4? LOS 4 NIVELES EN LA PRÁCTICA

LISTA DE VERIFICACIÓN DEL MODELO C4

DIAGRAMAR VS MODELAR

MODERNIZACIÓN DE UN E-COMMERCE

EN RESUMEN

ACERCA DE ICEPANEL

Fundamos IcePanel en 2020 por frustración con los productos existentes.
Productos que frenaban a los equipos en lugar de hacerlos avanzar.
Productos aburridos en lugar de agradables.
Productos que te obligaban a usar, no que te entusiasmaba usar.
Productos optimizados para vender, no construidos para nosotros.

Comunicar sistemas complejos es más difícil que construirlos.
Diseñar el futuro se siente anclado en el pasado.
Lento, fragmentado y lleno de malentendidos.
Un paso adelante, dos pasos atrás.

Así que hicimos lo que hacen los constructores y lanzamos IcePanel. Aún estamos empezando, enfocados en crear la mejor herramienta para arquitectos.

Si te interesa la arquitectura, probablemente hayas visto a Luca en LinkedIn, YouTube o

en alguna conferencia. Tiene un don para explicar temas complejos de forma fácil de entender. Era la persona ideal cuando buscábamos a alguien que escribiera un ebook sobre el modelo C4. Admiramos el trabajo de Luca desde hace mucho tiempo y nos entusiasma colaborar con él para dar vida a este libro.

¡Feliz lectura y mantente tranquilo!

C A P Í T U L O

¿POR QUÉ EL MODELO C4?

¿POR QUÉ EL MODELO C4?

Todo sistema sociotécnico exige una perspectiva holística, que integre la comprensión de los requisitos del negocio, las características arquitectónicas y la preparación para la implementación antes de escribir una sola línea de código.

Los sistemas de software efectivos no nacen de componentes individuales aislados; surgen de una visión clara de cómo el sistema debe comportarse, evolucionar y dar soporte al contexto del negocio.

Un modelo arquitectónico compartido se vuelve esencial, no solo para alinear a las partes interesadas, sino también para orientar las decisiones de diseño con intención y precisión.

Los lenguajes de modelado tradicionales como UML se diseñaron pensando en el rigor y la formalidad, pero su complejidad a menudo los vuelve poco prácticos en entornos ágiles y de ritmo acelerado.

Por ejemplo, un diagrama de despliegue UML puede intentar capturar cada servidor, base de datos y conexión de red de un sistema, pero el nivel de detalle rápidamente se vuelve abrumador y difícil de mantener. Como resultado, muchos equipos abandonan estos modelos y prefieren bocetos rápidos de “cajas y líneas”, diagramas fáciles de dibujar pero propensos a inconsistencias, nombres ambiguos y niveles de abstracción mezclados.

Estos compromisos pueden ser convenientes a corto plazo, pero, en última instancia, debilitan la comprensión compartida que exige la arquitectura.

MODELO C4

El modelo C4 resuelve estas deficiencias ofreciendo un conjunto de abstracciones consistentes y por capas: Contexto, Contenedores, Componentes y Código. Estos niveles permiten a los equipos acercarse y alejarse del sistema de forma similar a como Google Maps te permite pasar de una vista general de una ciudad al detalle a nivel de calle.
En palabras de su creador, Simon Brown, el modelo C4 consiste en crear “mapas de tu código”: diagramas que muestran tanto dónde vive el sistema dentro de su entorno como la forma en que encajan sus piezas internas.

Este enfoque por capas no es solo visual: es comunicativo.

Los ejecutivos y las partes interesadas no técnicas pueden comprender de inmediato el contexto general, como quién usa el sistema y por qué. Los arquitectos y desarrolladores pueden profundizar en los contenedores y componentes, explorando la estructura y las responsabilidades del sistema. Cuando es necesario, normalmente durante revisiones de diseño o la planificación del desarrollo, una vista a nivel de código, como un diagrama de clases UML, ofrece precisión donde la precisión importa.

Al adaptar el nivel de abstracción a la audiencia, el modelo C4 garantiza claridad para cada grupo de interesados sin sacrificar la consistencia ni la trazabilidad. Reduce la ambigüedad, promueve una documentación más efectiva y fomenta un vocabulario arquitectónico que abarca el negocio, el diseño y la implementación.

LOS NIVELES
QUÉ CUBRE ESTA GUÍA

Esta guía presenta el modelo C4 como un marco práctico para visualizar y comunicar la arquitectura de software.

Explica cómo sus cuatro niveles —Contexto, Contenedores, Componentes y Código— ayudan a los equipos a razonar sobre los sistemas en distintos niveles de abstracción, manteniendo la consistencia y la claridad en todas las vistas.

A través de la modernización de una plataforma de e-commerce, la guía muestra cómo aplicar el modelo C4 en la práctica, destacando buenas prácticas, errores comunes y los beneficios de usar herramientas de modelado como IcePanel para mantener los diagramas precisos, conectados y fáciles de evolucionar.

En conjunto, estas ideas muestran cómo el modelo C4 convierte la documentación de arquitectura en una comprensión viva y compartida entre los equipos.

C A P Í T U L O

¿QUÉ ES EL MODELO C4?

¿QUÉ ES EL MODELO C4?

El modelo C4 es un marco ligero para visualizar la arquitectura de los sistemas de software. Su propósito no es reemplazar las especificaciones detalladas ni los diagramas de bajo nivel, sino ofrecer una forma clara y consistente de comunicar la estructura en distintos niveles de abstracción.

El modelo se basa en una idea simple: un sistema de software puede entenderse como un conjunto de “mapas”, cada uno con un nivel de detalle distinto.

En el nivel más alto, ves cómo encaja el sistema en su entorno. A medida que te acercas, descubres los bloques internos, sus responsabilidades y sus relaciones. Este enfoque por capas evita los problemas de intentar expresarlo todo en un único diagrama, manteniendo a la vez las distintas vistas consistentes y conectadas.

LOS NIVELES EXPLICADOS

Las cuatro capas del modelo son:

Contexto del sistema: muestra el sistema en su entorno, destacando a los usuarios y las dependencias externas.

Contenedores: identifican los bloques tecnológicos de alto nivel (unidades ejecutables y desplegables) del sistema, como aplicaciones, bases de datos o intermediarios de mensajes.

Componentes: describe cómo un contenedor se estructura internamente en módulos o controladores y cómo estos colaboran.

Código: ofrece un primer plano del nivel de implementación, normalmente con diagramas de clases o funciones, donde se necesita detalle.

Esta progresión crea una forma estructurada de “acercarse” desde una vista general del sistema hasta los aspectos detallados de la implementación. El valor del modelo no está en producir diagramas exhaustivos, sino en asegurar que cada nivel se comunique con claridad a su audiencia prevista.

Los ejecutivos y las partes interesadas del producto pueden centrarse en el contexto y los contenedores, mientras que los arquitectos y desarrolladores pueden apoyarse en los componentes y el código para las discusiones de diseño técnico.

Al ofrecer la formalidad justa para mantener la consistencia, y a la vez ser lo bastante ligero para resultar práctico, el modelo C4 cierra la brecha entre los bocetos informales improvisados y la rigidez de los lenguajes de modelado tradicionales.

C A P Í T U L O

LOS 4 NIVELES EN LA PRÁCTICA

LOS 4 NIVELES EN LA PRÁCTICA

La fortaleza del modelo C4 está en su capacidad para describir un sistema en distintos niveles de abstracción, según la audiencia y el propósito. Cada nivel se centra en una historia específica y ofrece un diagrama que complementa a los demás sin duplicarlos.

NIVEL 1: CONTEXTO

El diagrama de contexto del sistema representa el sistema de software como una sola caja

dentro de su entorno, mostrando las personas, organizaciones y sistemas externos que interactúan con él.

En este nivel, el sistema se trata como un producto o servicio diferenciado, a menudo propiedad de un solo equipo que lo mantiene, que entrega valor a sus usuarios y que puede integrarse con otros sistemas.

Por ejemplo, una plataforma de venta en línea podría interactuar con clientes que navegan y compran productos, con personal de almacén que gestiona el despacho de pedidos y con una pasarela de pagos como Stripe.

Esta vista enfatiza el alcance y los límites, aclarando quién usa el sistema, de qué sistemas externos depende y cómo encaja en el entorno más amplio, sin entrar en detalles internos de implementación.

CONTEXTO

NIVEL 2: CONTENEDORES

Un diagrama de contenedores se acerca a un único sistema para mostrar las aplicaciones, servicios y almacenes de datos que lo componen, representando las decisiones tecnológicas de alto nivel y las unidades de despliegue. Cada contenedor es una aplicación o servicio que se puede desplegar y/o ejecutar por separado, con sus propias responsabilidades.

En el ejemplo de venta en línea, los contenedores podrían incluir una aplicación web, una app móvil, un backend de API, una base de datos relacional para los datos de productos y pedidos, y un intermediario de mensajes para coordinar tareas asíncronas como el envío de confirmaciones de pedido. Esta vista ayuda a los arquitectos y a los interesados técnicos a entender cómo está compuesto el sistema, cómo se distribuyen las responsabilidades y cómo interactúan los contenedores entre sí y con los sistemas externos.

CONTENEDORES

NIVEL 3: COMPONENTES

Un diagrama de componentes se acerca a un único contenedor para mostrar los bloques principales que colaboran para cumplir sus responsabilidades. Cada componente tiene un rol y una interfaz claramente definidos, lo que hace que el sistema sea más fácil de entender, mantener y evolucionar.

Por ejemplo, dentro del contenedor de API backend de una plataforma de venta en línea, el Servicio de Pedidos podría dividirse en componentes como el Controlador de Pedidos, que atiende las solicitudes entrantes; el Procesador de Pedidos, que gestiona la lógica de negocio y la validación; y el Adaptador de Pasarela de Pagos, que se comunica con proveedores de pago externos como Stripe.

Esta vista está pensada principalmente para arquitectos y desarrolladores, y ofrece un mapa claro de la estructura interna de un contenedor sin incluir detalles innecesarios para los interesados de más alto nivel.

COMPONENTES

NIVEL 4: CÓDIGO

El diagrama de código se acerca a un único componente para mostrar su implementación interna mediante clases, funciones u otras construcciones de programación. Captura cómo el componente cumple sus responsabilidades, aportando precisión donde se necesita para el desarrollo o las discusiones de diseño.

Por ejemplo, dentro del componente Servicio de Pedidos de una plataforma de venta en línea, un diagrama de código podría mostrar clases como Order, OrderItem y Customer, junto con los métodos que orquestan la validación de pedidos, el procesamiento de pagos y la actualización del inventario.

Este nivel está pensado principalmente para desarrolladores y ofrece una visión de la implementación del sistema manteniendo la alineación con los diagramas de nivel superior. Rara vez es necesario; muchas áreas de un sistema pueden detenerse en el nivel de componentes y crear diagramas de código solo donde se requiera claridad adicional. En las prácticas modernas, los diagramas de código se usan poco, salvo para la incorporación de personal, revisiones de diseño o sistemas altamente regulados que exigen documentación precisa.

CÓDIGO

ORIENTACIÓN SOBRE LOS NIVELES DE COMPONENTES Y CÓDIGO

Es importante señalar que los niveles 3 y 4 no siempre son necesarios para todos los sistemas ni para todas sus partes. En muchos casos, entender el sistema hasta el nivel de contenedores es suficiente para la mayoría de las discusiones, la planificación y la toma de decisiones. Los equipos pueden optar por crear diagramas de componentes solo para las áreas que requieren claridad adicional, como la lógica de negocio compleja o los servicios críticos.

BUENAS PRÁCTICAS Y CONSEJOS

Al usar el modelo C4, el objetivo es la claridad y la comunicación efectiva, no una

documentación exhaustiva. Las siguientes pautas pueden ayudar a los equipos a obtener el máximo valor de este enfoque:

Adapta el nivel de detalle a tu audiencia
No todos los interesados necesitan ver todos los niveles. Los ejecutivos y gerentes de producto suelen beneficiarse de los diagramas de contexto y de contenedores, mientras que los desarrolladores y arquitectos pueden necesitar los componentes y, ocasionalmente, vistas a nivel de código.

Dedica tiempo a la intención y la claridad
Antes de crear diagramas, piensa bien qué necesitas expresar. Reúne los requisitos, entiende a tu audiencia y tradúcelo en arquitectura.

Sin una intención clara, es difícil diseñar diagramas C4 efectivos en los distintos niveles.

Usa nombres y notación consistentes
Define convenciones claras para nombrar sistemas, contenedores y componentes.

Etiquetas, símbolos y colores consistentes mejoran la comprensión y reducen la ambigüedad entre diagramas.

Empieza por el contexto y los contenedores
Comienza con los diagramas de contexto del sistema y de contenedores para establecer límites, dependencias y responsabilidades de alto nivel. Profundiza en los componentes solo cuando se necesite más claridad. Los diagramas de código son opcionales y deben reservarse para áreas complejas o necesidades específicas.

Itera y mantén los diagramas
La arquitectura evoluciona con el tiempo. Trata los diagramas como documentos vivos y actualízalos para reflejar los cambios importantes. Evita dedicar demasiado tiempo a perfeccionar diagramas que quedan obsoletos rápidamente.

Aprovecha herramientas y plantillas
Usa herramientas de diagramación o plantillas que admitan capas y consistencia. Esto reduce el esfuerzo y ayuda a mantener un lenguaje visual estándar en toda tu documentación.

LISTA DE VERIFICACIÓN DEL MODELO C4

C A P Í T U L O

LISTA DE VERIFICACIÓN DEL MODELO C4

1

2 3 4 5

TODOS LOS OBJETOS TIENEN NOMBRES CLAROS, CON LAS SIGLAS DESARROLLADAS O EXPLICADAS.

EL DIAGRAMA TIENE UN NOMBRE CLARO Y DESCRIPTIVO.

LAS RESPONSABILIDADES DE LOS OBJETOS SON EVIDENTES O ESTÁN RESPALDADAS POR DESCRIPCIONES VISIBLES.

UNA BREVE DESCRIPCIÓN EXPLICA QUÉ REPRESENTA EL DIAGRAMA Y SU PROPÓSITO.

LAS RELACIONES ENTRE OBJETOS (CONEXIONES) ESTÁN ETIQUETADAS PARA INDICAR INTENCIÓN O FLUJO.

6

MEDIANTE UNA LEYENDA.

7

LAS FORMAS, ESTILOS DE LÍNEA, BORDES, PUNTAS DE FLECHA, ÍCONOS Y COLORES SE USAN DE FORMA CONSISTENTE Y CON SIGNIFICADO.

8

EL TAMAÑO DE LOS OBJETOS ES ADECUADO Y PROPORCIONAL A SU IMPORTANCIA O JERARQUÍA.

9

EL DIAGRAMA MUESTRA EL NIVEL DE DETALLE ADECUADO PARA LA AUDIENCIA PREVISTA.

10

LA NOTACIÓN Y LOS SÍMBOLOS ESTÁN EXPLICADOS, IDEALMENTE

LA AUDIENCIA PREVISTA PUEDE ENTENDER EL DIAGRAMA SIN EXPLICACIONES ADICIONALES.

C A P Í T U L O

DIAGRAMAR VS MODELAR

DIAGRAMAR VS MODELAR

¿Cuántas veces has abierto un diagrama en Confluence o Miro solo para darte cuenta de que estaba desactualizado, era inconsistente o le faltaban cambios clave?

Sucede todo el tiempo.

Diagramar es ideal para explicar una idea en el momento, pero en cuanto el sistema evoluciona, los diagramas dejan de reflejar la realidad. De pronto ya no solo explicas la

arquitectura: explicas las diferencias entre el diagrama y el sistema real.

Modelar, en cambio, construye una única fuente de verdad.
Una representación estructurada de sistemas, contenedores, componentes y relaciones. Este modelo se convierte en la base para actualizar automáticamente múltiples diagramas C4, garantizando que los cambios se propaguen a todas las vistas sin duplicaciones ni desfases.

Además, modelar permite a los equipos escalar muchos diagramas en toda la organización, imponiendo consistencia y asegurando que los objetos de los diagramas estén actualizados.

Imagina cambiar el nombre de un sistema externo en el diagrama de contexto. En una herramienta de diagramación pura, debes buscar y actualizar a mano cada diagrama donde aparece. En una herramienta de modelado, el cambio se hace una vez en el modelo y todos los diagramas lo reflejan al instante.

Además, el modelo permite a los equipos consultar la arquitectura, por ejemplo, identificar todos los componentes que dependen de un servicio concreto u obtener información sobre tecnologías obsoletas.
Modelar exige más esfuerzo inicial, como definir nombres, metadatos y estructura, pero se compensa con consistencia, facilidad de descubrimiento y agilidad.

Aplicado a C4, modelar te permite crear tu diagrama de contexto y luego refinarlo de forma fluida en contenedores, componentes y vistas de despliegue o dinámicas, todo conectado a la misma arquitectura subyacente.
Herramientas como **IcePanel** encarnan esta mentalidad de “modelar primero”. Permiten a los arquitectos capturar la arquitectura como modelos conectados y alineados con el modelo C4, garantizando que los diagramas se mantengan sincronizados, que las dependencias sean más fáciles de rastrear y que la arquitectura evolucione junto con el sistema.

Ahora pasemos de la teoría a **ejemplos reales** para que veas cómo funciona el modelo C4 en la práctica. Usaré IcePanel para recorrer un sistema y mostrarte cómo el modelado hace que la documentación de arquitectura sea confiable, consistente y fácil de evolucionar.

C A P Í T U L O

MODERNIZACIÓN DE UN E-COMMERCE

CONTEXTO Y ENFOQUE

Una empresa de e-commerce en crecimiento enfrentaba desafíos cada vez mayores con su plataforma monolítica heredada. Los lanzamientos se habían vuelto lentos y engorrosos, lo que dificultaba responder rápidamente a las necesidades de los clientes o implementar nuevas funcionalidades.

Con los años, el sistema se había convertido en una arquitectura fuertemente acoplada, con dependencias complejas entre módulos, lo que encarecía cada vez más el mantenimiento y el escalamiento. La organización quería aumentar su agilidad, reducir

la complejidad operativa y permitir que los equipos avanzaran más rápido, garantizando a la vez la estabilidad y la confiabilidad de las operaciones críticas del negocio.

Tras evaluar varios enfoques, la empresa decidió modernizarse con microservicios, aprovechando su experiencia interna en Node.js y desplegándolos en AWS.

Se eligió una arquitectura “serverless primero” para acelerar los lanzamientos, reducir la carga operativa del equipo de plataforma y permitir que los equipos trabajen con autonomía dentro de un modelo organizacional federado. Además, los desarrolladores aprovechan una infraestructura en la nube probada sin necesitar gran experiencia en plataforma, ganando velocidad y seguridad al entregar nuevas funcionalidades.

En este ejemplo, usaremos el modelo C4 para visualizar el nuevo sistema. Exploraremos distintas capas, desde el contexto hasta los componentes, omitiendo la vista de código, que rara vez se necesita.

Para ilustrar el proceso de migración, una parte del sistema se mostrará ya modernizada en microservicios, mientras que otra seguirá en el monolito. Cada capa incluirá ejemplos concretos para aclarar cómo aplicar el modelo C4 en la práctica, ayudando a los lectores a entender no solo cómo es la arquitectura, sino cómo comunicarla eficazmente a los distintos interesados.

EL NIVEL DE CONTEXTO

NIVEL DE CONTEXTO

El **nivel de contexto del sistema** es el nivel de abstracción más alto del modelo C4. En esta etapa, el sistema se representa como una “caja negra” y el foco no está en su interior, sino en cómo interactúa con el mundo que lo rodea. El objetivo es mostrar **quién usa el sistema** y **con qué otros sistemas se comunica**, estableciendo un límite claro de responsabilidades y dependencias. En este nivel, los diagramas suelen incluir:

Personas (actores): usuarios finales, clientes o roles que interactúan con el sistema.

Sistemas de software: aplicaciones o plataformas externas con las que se integra

el sistema, como pasarelas de pago, servicios de notificación o CRM.

El propio sistema: la caja central que representa el software que estamos construyendo o modernizando, como en nuestro caso.

En nuestro ejemplo de modernización del e-commerce, el diagrama captura exactamente estos elementos.

En el centro encontramos la nueva plataforma de e-commerce, que representa la parte modernizada del sistema, construida con microservicios y micro frontends.

A su alrededor, varios sistemas externos definen su entorno: Salesforce para la gestión de usuarios, Twilio para las notificaciones, Adyen para los pagos y un CMS SaaS para gestionar el catálogo de productos.
En el lado del despacho de pedidos, el diagrama muestra el sistema de fulfillment heredado, una aplicación monolítica que sigue gestionando los envíos y el procesamiento de pedidos.

A su lado se encuentra la plataforma de e-commerce heredada, que sigue en uso para las partes del negocio aún no migradas.

Al incluir estos sistemas, el diagrama ofrece una imagen realista del estado de transición: el nuevo sistema de e-commerce ya interactúa tanto con servicios externos modernos como con componentes monolíticos existentes.

AUDIENCIA Y CASOS DE USO

El nivel de contexto del sistema está diseñado para una comunicación amplia. Ofrece una vista lo bastante simple para que la sigan los interesados no técnicos, y a la vez lo bastante precisa para que los arquitectos la usen como base de un diseño más profundo.

> **Los ejecutivos y los interesados del negocio** se benefician de esta vista porque destaca el alcance y los límites. Pueden ver quién interactúa con el sistema, qué proveedores externos son críticos para su operación y cómo los sistemas heredados siguen formando parte del panorama. Esto les permite evaluar riesgos, planificar presupuestos y alinear las decisiones tecnológicas con la estrategia del negocio.
>
> **Los arquitectos y líderes técnicos** usan esta vista al inicio de las discusiones para delimitar los límites del sistema y sus dependencias externas antes de profundizar en contenedores o componentes. Así, todo el equipo parte de la misma comprensión de lo que el sistema es y lo que no es.

Este nivel es más valioso cuando necesitas dar un paso atrás y ver el panorama completo, como dice Simon Brown. En los sistemas distribuidos, las nuevas funcionalidades suelen abarcar varios dominios, lo que requiere conversaciones que alineen a los interesados de negocio y técnicos hacia el mismo objetivo. Un diagrama de contexto ofrece una vista compartida. También es útil al evaluar nuevas integraciones con terceros, ya que muestra cómo encajan los sistemas externos dentro de los límites y responsabilidades existentes. Por último, esta abstracción es especialmente eficaz para incorporar a nuevos miembros del equipo, dándoles una comprensión clara del alcance del sistema, sus usuarios y sus dependencias clave antes de que se sumerjan en los detalles de bajo nivel.

En nuestro caso, el diagrama de contexto se vincula directamente con los objetivos de negocio que impulsan la modernización. La apuesta de la empresa por la agilidad y lanzamientos más rápidos se refleja en la elección de servicios externos como Twilio, Salesforce y Adyen, que reducen la necesidad de construir estas capacidades internamente. La presencia continua de los sistemas heredados de fulfillment y e-commerce muestra cómo la organización gestiona la transición y el riesgo, asegurando que las operaciones críticas se mantengan estables mientras se entregan nuevas

capacidades.

En otras palabras, los límites trazados en el diagrama de contexto no son arbitrarios: reflejan las prioridades del negocio de entregar más rápido, reducir la carga operativa y tener un camino claro hacia la modernización sin interrumpir las ventas ni el despacho de pedidos en curso.

ERRORES COMUNES

Los equipos suelen cometer algunos errores recurrentes al crear diagramas de contexto del sistema. Uno frecuente es mezclar contenedores internos con el contexto del sistema; por ejemplo, mostrar APIs, bases de datos o servicios que pertenecen al interior de la plataforma.

> Esto difumina la abstracción y hace que el diagrama sea más difícil de interpretar.

Otro error es añadir un detalle técnico excesivo, como protocolos o decisiones de infraestructura, que distraen del propósito de este nivel. Por último, nombrar de forma inconsistente a los actores y sistemas, por ejemplo, alternar entre “Cliente” y “Usuario final” en el mismo diagrama, puede generar confusión y debilitar la comprensión compartida que el diagrama debe aportar. En concreto, este problema se resuelve fácilmente con herramientas de modelado como IcePanel, que ofrecen una forma sencilla de arrastrar y soltar elementos existentes en las distintas capas del modelo C4.

CONSEJOS PRÁCTICOS

Para aprovechar al máximo este nivel, procura que los diagramas sean simples y legibles, centrándote solo en personas, sistemas y sus relaciones. Etiqueta las relaciones con claridad, pero evita la jerga técnica que solo una parte de la audiencia entenderá. Recuerda que estos diagramas pueden ser útiles en presentaciones ejecutivas, sesiones de planificación estratégica y hojas de ruta de alto nivel, donde la claridad sobre los límites y las dependencias es lo más importante. Trátalo ante todo como una herramienta de comunicación: su fortaleza está en alinear a interesados diversos en torno a una visión compartida del lugar del sistema en su entorno.

NIVEL DE CONTENEDORES

Cuando pasamos del Contexto del Sistema al **nivel de Contenedores**, el modelo C4 nos pide representar los **principales bloques de ejecución** que forman nuestro sistema. Un contenedor es cualquier aplicación o almacén de datos que requiere su propio entorno de ejecución. Puede ser una aplicación web, una app móvil, un microservicio, una base de datos o incluso una dependencia SaaS externa.

Esta definición es crucial: un **contenedor no es un contenedor Docker**.

En la terminología de C4, la palabra “contenedor” proviene de la idea de “contener” código o datos. Cada contenedor es algo que puedes desplegar, operar y monitorear de forma independiente.

Esta vista es especialmente valiosa porque equilibra la **abstracción con la concreción técnica**. Un diagrama de contexto podría simplemente indicar que “el sistema de e-commerce se integra con un proveedor de pagos”, pero en el nivel de contenedores aprendemos *cómo* se hace.

SUBDOMINIO CATÁLOGO

El subdominio Catálogo es responsable de mostrar los productos a los clientes. En C4, representamos este subdominio como un grupo de contenedores, cada uno con un límite de ejecución claro:

Catalog MFE (Micro Frontend) Una aplicación frontend escrita en Node.js, desplegada de forma independiente y responsable de mostrar el catálogo de productos en el navegador. En C4, se representa como un contenedor de tipo aplicación, anotado con la tecnología (Node.js) y su responsabilidad (“Provee el catálogo de productos”).

Catalog Backend Una API de estilo backend-for-frontend (BFF) que obtiene los productos y los entrega al frontend. De nuevo, es un contenedor separado, desplegado en su propio entorno de ejecución, que expone una API JSON/HTTPS.

Catalog Database Un almacén de datos persistente, modelado como contenedor porque se ejecuta de forma independiente y tiene un entorno de ejecución claro (NoSQL + DynamoDB). En el modelo C4, las bases de datos son ciudadanos de primera clase: se tratan como contenedores porque requieren consideraciones de

despliegue, escalamiento y operación. Las anotamos con su tecnología (MongoDB, DynamoDB) y su propósito (“Almacena los metadatos de los productos y referencias a recursos multimedia como videos o imágenes”).

Catalog Metadata Backend Un servicio responsable de sincronizar la información de productos desde un CMS externo (Contentful) y enviar las actualizaciones a la Catalog Database. También reenvía datos al SaaS de Personalización. Es un ejemplo de manual de un contenedor de microservicio: se ejecuta de forma independiente, tiene una API bien definida y una responsabilidad clara.

Al modelar cada uno de estos elementos como contenedores, hacemos visibles no solo las piezas del catálogo, sino también sus interacciones: el MFE depende del Backend, que consulta la Base de Datos; el Metadata Backend sincroniza datos desde proveedores SaaS externos.

SUBDOMINIO CHECKOUT

El subdominio Checkout gestiona la creación de pedidos y el procesamiento de pagos. De nuevo, cada contenedor es una unidad de ejecución:

Checkout MFE Un contenedor frontend que provee la interfaz de usuario del checkout. Orquesta las llamadas al backend y al proveedor de pagos externo.

Checkout Backend La API para crear un pedido. Se representa como contenedor porque es un servicio Node.js desplegable que expone una interfaz HTTPS.

Payment Backend Un contenedor dedicado a gestionar el procesamiento de pagos. Abstrae la integración con Adyen y guarda los comprobantes. En C4, es una buena práctica separar estas responsabilidades en sus propios contenedores para que puedan evolucionar de forma independiente.

Orders Database Otro contenedor de base de datos, implementado con DynamoDB y DynamoDB Streams, que almacena los detalles de los pedidos y emite cambios para los procesos posteriores (p. ej., el despacho de pedidos).

Payments Database Una tabla de DynamoDB con IDs de comprobantes, fechas, montos y otros metadatos de pago. Se anota como contenedor de datos, con una responsabilidad clara y su etiqueta de tecnología.

Observa cómo esta vista aclara que el flujo de checkout no es un único backend

monolítico, sino un conjunto de contenedores que cooperan, cada uno con su propio entorno de ejecución y responsabilidad.

POR QUÉ IMPORTA EL NIVEL DE CONTENEDORES

El diagrama de contenedores es especialmente útil para audiencias que necesitan una comprensión técnica más profunda sin ahogarse en detalles de implementación:

Arquitectos e ingenieros sénior: les ayuda a razonar sobre los límites del sistema, identificar responsables y asegurar que la arquitectura soporte capacidades de negocio como la escalabilidad, la resiliencia o el cumplimiento normativo.

Desarrolladores nuevos en el sistema: sirve como mapa, aclarando dónde están las responsabilidades, qué servicios existen y cómo fluyen los datos entre ellos. En lugar de un diagrama plano con decenas de cajas, los contenedores aportan estructura mediante agrupaciones lógicas.

Product owners e interesados técnicos: al vincular los contenedores con subdominios y equipos, pueden entender mejor cómo se dividen las responsabilidades y cómo encajan los servicios externos en el sistema.

En este nivel, uno de los beneficios clave es **hacer explícitas las compensaciones (trade-offs)**. Por ejemplo, persistir los pedidos en DynamoDB con streams conectados a EventBridge no es solo una decisión técnica de implementación; expresa prioridades como escalabilidad, confiabilidad y consistencia eventual. Del mismo modo, introducir un búfer mediante colas SQS muestra cómo la arquitectura anticipa un tráfico alto sin saturar los servicios posteriores.

LOS CONTENEDORES COMO BASE PARA LOS EQUIPOS

Por último, el nivel de contenedores tiene una dimensión organizacional. Al estructurar la plataforma de e-commerce en subdominios y asociar un contenedor o grupo de contenedores con equipos, hacemos explícita la correspondencia entre equipos y arquitectura. Esto es especialmente importante en las arquitecturas modernas de microservicios y micro-frontends, donde la Ley de Conway nos enseña que la estructura de los equipos moldea el diseño del sistema.

Poder señalar un contenedor y decir “esto es responsabilidad del equipo de Catálogo” aclara la propiedad y evita ambigüedades. También respalda prácticas como Team Topologies, donde los límites entre equipos y servicios deben diseñarse deliberadamente.

BUENAS PRÁCTICAS Y ORIENTACIÓN PARA

MODELAR CONTENEDORES

Al trabajar en el nivel de contenedores, es fácil simplificar demasiado (dibujar solo un par de cajas) o complicar demasiado (convertir el diagrama en un mapa de cableado). Las siguientes prácticas ayudan a lograr el equilibrio adecuado:

Usa la capacidad de despliegue como límite Un contenedor representa algo que puede desplegarse, escalarse y operarse de forma independiente. Si puedes ejecutarlo en su propio proceso, máquina virtual, contenedor o entorno gestionado, es candidato a ser un contenedor en tu diagrama. Esta pauta evita modelar en el nivel de abstracción equivocado.

Haz explícitas las responsabilidades Cada contenedor debe indicar en lenguaje sencillo qué hace. “Provee el catálogo de productos”, “Almacena los detalles de los pedidos” o “Procesa los pagos” son claros y accesibles incluso para lectores no técnicos. Evita etiquetas puramente técnicas como “Servicio A” o “BD1”.

Anota siempre la tecnología El modelo C4 recomienda emparejar responsabilidades con tecnologías: Node.js – Provee el catálogo de productos, DynamoDB – Almacena los detalles de los pedidos. Esto ayuda a los lectores a relacionar rápidamente lo que hace algo con cómo está implementado. Con el tiempo, esta práctica también hace transparentes las migraciones o las decisiones tecnológicas.

Trata las bases de datos como ciudadanos de primera clase En muchas notaciones de diagramación, los almacenes de datos quedan relegados a un papel secundario. En C4, las bases de datos son contenedores porque tienen comportamiento en ejecución, necesidades de escalamiento e impacto operativo. Modelarlas explícitamente revela compensaciones (p. ej., NoSQL vs relacional, streams vs colas) que a menudo tienen relevancia arquitectónica.

Muestra los sistemas externos con la misma claridad Las integraciones SaaS (p. ej., Adyen para pagos, Contentful como CMS o un motor de personalización) deben modelarse como cualquier otro contenedor. Aunque no las construyas ni despliegues, afectan la disponibilidad, la resiliencia y las responsabilidades del equipo.

Agrupa los contenedores por subdominio o por equipo responsable Agrupar los contenedores en subdominios (como Catálogo, Checkout, Fulfillment) aclara las capacidades de negocio detrás de la arquitectura. Asignar estos grupos a equipos fortalece la alineación entre arquitectura y organización, un principio clave en el diseño moderno de sistemas distribuidos.

MÁS QUE UNA IMAGEN ESTÁTICA

Una de las funciones más potentes de las herramientas modernas, como IcePanel en nuestro caso, es la capacidad de ir más allá de los diagramas estáticos. La función **Flow** permite ilustrar secuencias de interacciones, como un cliente navegando por el catálogo, realizando un pedido y procesando el pago. Esta vista interactiva reúne lo que normalmente se divide entre diagramas de arquitectura y diagramas de secuencia, permitiendo a los arquitectos “reproducir” un escenario paso a paso.

Por ejemplo, el flujo de checkout comienza cuando el cliente invoca el Micro Frontend de Checkout, que a su vez se comunica con el Checkout Backend para generar el ID del pedido. Los datos de pago se gestionan a través de Adyen, los comprobantes se escriben en la Payments Database y los pedidos se guardan en la Orders Database. A partir de ahí, se activan procesos posteriores como el despacho de pedidos y las notificaciones al cliente.

Explicar un diagrama en una reunión con este tipo de funciones puede simplificar drásticamente las discusiones. En lugar de recorrer un dibujo estático, puedes “reproducir” las interacciones y dejar que el diagrama cuente la historia. Incluso fuera de las reuniones, basta con compartir el enlace a un diagrama del modelo C4 con una persona recién incorporada para que recorra las interacciones del sistema a su propio ritmo. Esto no solo acelera la incorporación, sino que aporta claridad sin que alguien tenga que estar presente para explicar la arquitectura.

EL NIVEL DE COMPONENTES

NIVEL DE COMPONENTES

El diagrama de nivel de componentes del modelo C4 se acerca al interior de un contenedor para ilustrar los bloques principales que lo forman y cómo colaboran. Cada componente se define por sus responsabilidades y límites, más que por su implementación técnica. En la práctica, un componente puede corresponder a un módulo, una clase o un conjunto cohesivo de funciones que, en conjunto, aportan parte del comportamiento del contenedor. El énfasis está en mostrar cómo encajan las piezas para cumplir el propósito del contenedor, sin un detalle abrumador.

CUÁNDO USARLO

No todo contenedor justifica un desglose a nivel de componentes. Para servicios simples, una vista a nivel de contenedor puede bastar para comunicar sus responsabilidades. Sin embargo, **para áreas complejas o críticas para el negocio**, la vista de componentes puede ser invaluable. Algunos ejemplos:

Un **sistema de gestión de pedidos**, donde múltiples flujos de trabajo (validación de pedidos, pago, envío) interactúan de formas sutiles.

Un **servicio de autenticación**, que puede incluir componentes para el almacenamiento de credenciales, la generación de tokens, la validación multifactor y la auditoría.

Un **backend de catálogo**, donde la personalización, la categorización y los detalles de los productos deben integrarse en una API cohesiva.

La regla general: usa diagramas de componentes donde la claridad adicional ayude a reducir malentendidos, acelerar la incorporación o resaltar compensaciones arquitectónicas.

¿A QUIÉN LE INTERESA?

La audiencia de los diagramas de nivel de componentes son **principalmente desarrolladores y arquitectos.**

Los desarrolladores se benefician de entender la estructura interna de un servicio antes de sumergirse en el código. Los arquitectos usan estos diagramas para razonar sobre las decisiones de diseño: si las responsabilidades están bien separadas, las dependencias bien gestionadas y los límites claramente definidos. Los product owners o interesados no técnicos normalmente no necesitan este nivel de detalle.

EJEMPLO: CATALOG BACKEND

Considera un servicio backend de catálogo que impulsa la interfaz del catálogo de una plataforma de e-commerce. Este contenedor debe:

Obtener datos de la base de datos de productos.
Interactuar con un sistema de personalización externo para adaptar los resultados. Exponer APIs para distintas vistas del catálogo: productos más nuevos, más vendidos, recomendaciones personalizadas y páginas de detalle de producto.

En el nivel de componentes, podríamos modelar los siguientes bloques:

CatalogController Atiende las solicitudes de API entrantes desde la interfaz.
CatalogService Orquesta las solicitudes delegando en el repositorio y en el cliente de personalización, aplicando reglas de negocio y dando forma a las respuestas.
ProductRepository Provee acceso a los datos de productos almacenados en la base de datos.
PersonalizationClient Se integra con el sistema de personalización externo.

Esta vista destaca no solo la existencia de un contenedor, sino las responsabilidades y la colaboración de sus partes internas, facilitando la incorporación de nuevos desarrolladores o el análisis de cambios futuros.

BUENAS PRÁCTICAS

Para sacar el máximo provecho de los diagramas de nivel de componentes, es importante lograr un equilibrio entre detalle y claridad. Los componentes deben documentarse de forma selectiva, centrándose solo en las áreas del sistema donde la complejidad hace valiosa una estructura adicional. Los diagramas deben seguir siendo conceptuales, enfatizando responsabilidades y relaciones en lugar de ahogarse en detalles de implementación.

Mantener un número manejable de componentes por vista asegura que el diagrama siga siendo legible; si un servicio crece demasiado, es mejor crear subdiagramas que amontonarlo todo en una sola imagen.

Las convenciones de nombres también juegan un papel clave: alinear los nombres de los componentes con conceptos del dominio, como *ProductRepository* o *CatalogService*, hace que los diagramas sean intuitivos tanto para audiencias técnicas como de negocio.
Por último, los diagramas de nivel de componentes deben tratarse como guías de la intención de diseño, más que como reflejos exactos del código. Cuando se requiere

mayor precisión técnica, complementar el diagrama con referencias directas a los repositorios de código permite mantenerse actualizado sin sobrecargar la representación visual.

EL NIVEL DE CÓDIGO

NIVEL DE CÓDIGO

El diagrama de nivel de código es el paso más detallado del modelo C4. Profundiza en un único componente para mostrar la estructura interna de clases, interfaces, métodos o funciones. En este punto, el diagrama se parece a un diagrama de clases UML o a una vista de paquetes del código fuente. Su propósito es dar a los desarrolladores una referencia concreta de cómo se organiza el interior de un componente.

Por ejemplo, considera un componente *CheckoutService* responsable de orquestar el carrito de compras y el flujo de pago. Un diagrama de nivel de código podría mostrar las clases *OrderValidator*, *PaymentProcessor* y *ReceiptGenerator*, destacando sus responsabilidades y dependencias. Esto ofrece un mapa explícito de las piezas en movimiento dentro del componente.

A pesar de su claridad, **el nivel de código se usa poco**. En la mayoría de los casos, los desarrolladores prefieren consultar directamente el código fuente, y mantener estos diagramas sincronizados con bases de código que evolucionan rápidamente puede volverse tedioso. Aun así, hay momentos en que son valiosos: por ejemplo, en dominios críticos para la seguridad, en procesos de incorporación donde hay que explicar rápidamente la complejidad interna, o al documentar frameworks y librerías para consumidores externos.

Una forma práctica de hacer útil este nivel sin generar carga de mantenimiento es enlazar los diagramas al repositorio del código fuente. En lugar de congelar la estructura del código en imágenes estáticas, referencia el repositorio de GitHub donde vive el código. Así los diagramas siguen siendo ligeros y los lectores tienen un camino claro hacia los detalles de implementación más recientes.
Una simple anotación como “Ver repositorio de GitHub: /checkout-service” mantiene la referencia viva y confiable.

En resumen, el nivel de código conviene usarlo de forma selectiva, por parte de arquitectos y desarrolladores sénior, cuando hay una necesidad real de explicar detalles internos. Combinado con enlaces a repositorios vivos, puede aportar claridad sin añadir una carga de mantenimiento innecesaria.

C A P Í T U L O

EN RESUMEN

EN RESUMEN

El modelo C4 nos da una forma estructurada de contar la historia de nuestra arquitectura de software, desde el contexto general del sistema hasta los detalles del código. Al refinar la vista progresivamente, nos comunicamos eficazmente con cada

audiencia: el negocio en el nivel de contexto, desarrolladores y arquitectos en contenedores y componentes, y ocasionalmente ingenieros en el nivel de código.

En todos estos diagramas, el valor clave está en la claridad.

En lugar de abrumar a los equipos con abstracciones vagas o código de bajo nivel, C4 aporta el detalle justo para iluminar responsabilidades, límites e interacciones.

Combinado con prácticas como el Diseño Guiado por el Dominio (DDD) y la asignación explícita de contenedores a equipos, este enfoque ayuda a crear arquitecturas comprensibles y capaces de evolucionar. Por supuesto, los diagramas solo valen lo que valga su mantenimiento.

En organizaciones que avanzan rápido, los dibujos estáticos quedan obsoletos pronto y erosionan la confianza. Aquí entran las herramientas modernas. Plataformas como **IcePanel** llevan el modelo C4 más allá del papel o las diapositivas: mantienen los diagramas sincronizados con la realidad, permiten explorar los flujos del sistema de forma interactiva y simplifican las reuniones o sesiones de incorporación. Las vistas de dependencias, los flujos y los enlaces en vivo convierten los diagramas en documentación viva, no en entregables de un solo uso.

Si realmente quieres mejorar la forma en que tus equipos comunican la arquitectura, ahora es el momento de probar una herramienta que haga práctico el modelo C4. Ya sea que estés escalando un ecosistema de microservicios, coordinando micro-frontends o simplemente incorporando nuevos desarrolladores, contar con diagramas vivos e interactivos puede marcar la diferencia entre la confusión y la claridad.

Empieza a experimentar con C4 en tus propios sistemas y considera adoptar herramientas como IcePanel para dar vida a tus diagramas. Cuanto antes hagas tu arquitectura visible, compartida y mantenida de forma continua, más resilientes y alineados serán tus equipos.

El modelo C4 fue creado por Simon Brown y está licenciado bajo una licencia Creative Commons Atribución 4.0 Internacional.